VR Interview Simulation: A Resume-Grounded, Sequential Multi-Persona Conversational AI Framework for Immersive Interview Practice

N. Swetha
Department of Computer Science and Design
Rajalakshmi Engineering College
Chennai, Tamil Nadu, India
nslswetha2007@gmail.com

VA. Mukesh
Department of Computer Science and Design
Rajalakshmi Engineering College
Chennai, Tamil Nadu, India
mukeshva95@gmail.com

Under the guidance of Dr. P. Revathy

Abstract*—Mock-interview practice is constrained by the limited availability of experienced interviewers, generic questioning detached from a candidate's background, flat screen-based interfaces and little objective feedback on non-verbal conduct. Grounded in an initial human-centred design thinking empathy study of engineering candidates and placement faculty under the MS Innovate framework, this paper presents VR Interview Simulation, an immersive mock-interview platform that integrates a Unity virtual boardroom (OpenXR with a desktop fallback), resume-grounded conversational AI, a sequential three-interviewer panel, head-pose-based behavioural telemetry, an automated post-interview feedback engine, and an institutional placement dashboard. Candidate PDF resumes are parsed locally by a Python bridge (pypdf with heuristic pattern matching) into a structured profile, which is injected as runtime context into cloud-hosted Convai characters. A finite-state orchestrator moves the candidate through human-resources (HR), behavioural and hiring-manager stages (200 s per stage, 600 s total) and forwards earlier transcripts and dynamic follow-up context to subsequent interviewers. Headset orientation is sampled at 1 Hz to estimate active interviewer gaze alignment (within an empirical 35° cone), rotational posture stability, and response latency. A ten-competency evaluation model converts transcripts and telemetry into an on-screen summary, a ReportLab PDF report, and REST API synchronization to a centralized placement portal. The paper describes the empathy study, architecture, algorithms, and implementation-level verification. It does not report large-scale clinical trials: the scoring model is an engineering heuristic, gaze alignment is estimated from head pose rather than eye tracking, and while design thinking empathy maps and institutional performance data are documented, raw individual survey response sheets are unavailable.

***Keywords*—virtual reality, mock interview, conversational AI, resume grounding, multi-agent orchestration, head-pose gaze estimation, automated feedback, Unity, Convai**

I. INTRODUCTION

The employment interview remains a central step in candidate selection, and the design of structured interviews has attracted a substantial body of empirical research [1]. For candidates, the interview is also a performance: they must present their background coherently, answer behavioural questions under evaluative pressure, and manage visible conduct such as eye contact and pacing. Practising this performance with experienced interviewers is resource-intensive, and computer-based practice tools have therefore been explored for some time [2], [3].

Existing computer-based tools address parts of the problem. Interview simulators with a virtual interviewer provide repeated practice and automated feedback [2], [3], and a separate line of work analyses recorded interviews to support recruiters [4], [5]. What remains less common in the works reviewed here is a single practice environment that places the candidate in a three-dimensional interview setting, grounds the questioning in the candidate's own resume, presents a multi-person panel, observes non-verbal conduct from headset tracking, and returns structured feedback after the session. This paper describes *VR Interview Simulation*, a system that integrates these elements, and reports its design and implementation.

A. Problem Statement and Human-Centred Design Study

The project originates from an initial human-centred empathy study and design thinking inquiry conducted under the institutional MS Innovate framework at Rajalakshmi Engineering College. The study examined the placement-interview ecosystem across two primary user personas: (i) an engineering student candidate preparing for campus recruitment (represented by persona Monisha), and (ii) a placement cell faculty coordinator managing institutional hiring drives (represented by persona Sherin). For students, the empathy analysis identified acute evaluative anxiety, fear of technical probing and negative assessment, and a heavy reliance on passive YouTube preparation. Although candidates routinely practice questions in isolation, they lack realistic mock-interview exposure, access to experienced evaluators, and company- or resume-grounded questioning. For placement faculty, the core challenge is scale and heterogeneity: cohorts exhibit diverse competency levels, making standardized preparation ineffective, while severe faculty time constraints preclude delivering individual 1-on-1 mock interviews to hundreds of students. Furthermore, static aptitude tests fail to capture verbal composure, eye contact, or workplace readiness. Synthesizing these findings through a Pain & Gain analysis revealed clear dual-stakeholder requirements: students need an immersive, low-stakes practice space that reduces anxiety through resume-grounded simulation, while placement departments require objective performance analytics and cohort tracking. This inquiry led to the central design question: How might we design an AR/VR-based mock interview platform that helps students practice realistic interviews, reduce nervousness, and build confidence while enabling placement faculty to track performance and provide meaningful feedback? Conventional practice suffers from

five core constraints: (i) limited access to experienced evaluators; (ii) generic questioning detached from qualifications; (iii) flat 2D interfaces lacking social presence; (iv) the absence of multi-interviewer panel dynamics; and (v) the lack of objective non-verbal feedback and institutional cohort visibility [5].

B. Overview of the System

VR Interview Simulation is implemented as a Unity 6 application with an OpenXR-based virtual-reality mode and a desktop fallback. A candidate uploads a PDF resume, which a local Python bridge converts into a structured profile. The profile is held in a session manager and supplied as runtime context to three cloud-hosted conversational characters from the Convai platform. A deterministic finite-state orchestrator moves the candidate through a human-resources (HR) stage, a behavioural stage and a hiring-manager stage (200 s per stage, totaling 600 s / 10 minutes), forwarding earlier transcripts and dynamic follow-up context to later interviewers. During the session, the application samples headset orientation at 1 Hz to derive active gaze alignment, rotational posture stability and response latency from head-mounted-display (HMD) pose. Afterwards, a ten-competency evaluation model produces an on-screen summary, a ReportLab vector PDF report, and dispatches the session payload to a centralized Placement Data System (PostgreSQL database and Next.js web dashboard) via REST API.

C. Contributions and Organisation

The engineering contributions, each supported by the project inspection summarised in Section XIII, are:

- • an empathy-driven dual-stakeholder architecture combining an immersive interview environment, cloud conversational AI, local resume processing, and a centralized placement dashboard (Sections V, VIII);
- • resume-driven candidate-context extraction and dynamic prompt injection into conversational characters (Section VI);
- • a sequential three-interviewer orchestration across three 200 s stages with cross-stage transcript forwarding (Section VII);
- • 1 Hz HMD-based behavioural telemetry that estimates gaze alignment (35° cone), rotational posture stability, and response latency without external sensors (Section IX); and
- • an automated ten-competency evaluation pipeline that ends in an exported PDF placement report and institutional dashboard sync (Section X).

Section II reviews related work, Section III states the research gap, and Section IV states the objectives. Sections V to X describe the architecture and subsystems, Section XI the implementation, and Sections XII and XIII the testing and verification. Sections XIV to XVI cover limitations, future work and conclusions. No human-subject evaluation has been conducted; claims are therefore limited to what the system implements.

II. RELATED WORK

A. Interview Training Systems

Virtual Reality Job Interview Training (VR-JIT) provides repeated interviewing trials with a virtual human-resources representative together with online didactic material; a study of 70 participants reported preliminary support that participation was associated with enhanced vocational outcomes, including job offers in the following six months [2]. MACH (My Automated Conversation coacH) uses an animated character that asks interview questions, mimics some of the user's behaviour, shows non-verbal behaviour of its own and gives visual feedback after the interaction; in a week-long trial with 90 undergraduates, human experts rated the MACH group as improved in overall interview performance relative to control groups [3]. Both are evidence that simulated interviewers can support practice. Neither description in the cited abstracts mentions a head-mounted immersive environment, resume-derived questioning or a multi-person panel.

B. Automated Interview Analysis

HireNet predicts recruiter-assessed hirability from asynchronous video interviews, using a corpus of more than 7,000 candidates and modelling verbal, audio and video content [4]. The system is recruiter-facing and non-interactive. The survey of Qin et al. situates such work within talent analytics, covering resume understanding, interview-question recommendation and assessment scoring, and highlights fairness, interpretability and data-privacy concerns [5]. These concerns apply equally to the scoring component described in Section X.

C. Resume Understanding

Resume understanding converts resume files into structured fields. According to the survey in [5], early approaches used keyword search and rule-based matching, later ones used statistical sequence models, and recent ones treat the task as named-entity recognition with pretrained language models and layout-aware models. The parser in this work belongs to the first category (Section VI) and is acknowledged as the least capable of these.

D. Presence, Realism and VR in Affective Settings

Slater distinguishes Place Illusion (the sensation of being in the depicted place) from Plausibility Illusion (the credibility of the depicted events) and argues that, when both occur, participants tend to respond realistically to immersive virtual environments [6]. A meta-analysis of 13 studies reported a large effect of virtual reality exposure therapy relative to control conditions for anxiety disorders [7]. This evidence concerns clinical exposure therapy and is cited only as background on VR as a medium for eliciting realistic affective responses; it is not evidence about interview training.

E. Head Orientation as a Gaze Cue

Stiefelhagen and Zhu examined head orientation and eye orientation as contributors to gaze direction in four-person meetings and reported that head orientation contributed on average 68.9% to the overall gaze direction and supported focus-of-attention estimation with 88.7% average accuracy [8]. Their setting differs from a candidate facing a panel, but the result motivates head pose as a proxy for attention while making clear that it is a proxy and not a measurement of eye direction.

TABLE I
QUALITATIVE CAPABILITY COMPARISON

Approach	HMD VR	Conv. agent	Resume ground.	Multi-intvw.	Non-verbal	Feedback
VR-JIT [2]	NR	Yes	NR	No	NR	NR
MACH [3]	NR	Yes	NR	NR	Yes	Yes
HireNet [4]	No	No	No	No	Yes	No
This work	Yes	Yes	Yes*	Yes†	Yes‡	Yes§

NR: not reported in the cited abstract. Entries for prior work reflect only what the cited sources state and are not exhaustive descriptions of those systems. For this work, "Yes" denotes implementation, not validated effectiveness. *Heuristic parser. †Sequential, not simultaneous. ‡Head-pose estimate, not eye tracking. §Heuristic scoring, unvalidated.

III. RESEARCH GAP AND MOTIVATION

The works above treat interview practice, interview analysis and resume parsing largely as separate problems. The gap addressed here is one of integration: the proposed system combines (1) an immersive virtual interview setting, (2) conversational AI interviewers, (3) candidate resume grounding, (4) a sequential multi-persona panel with context propagation, (5) non-verbal observation from HMD tracking, and (6) automated post-interview feedback in one end-to-end pipeline. The claim made is that this combination is implemented and inspectable; it is not claimed that the system is the first of its kind, nor that the combination improves interview outcomes. Testing that proposition requires user studies that are outside the scope of this paper (Section XV).

IV. SYSTEM OBJECTIVES AND CONTRIBUTIONS

The project's engineering objectives, derived from the human-centred empathy study and implemented codebase, are to: (1) provide an immersive 3D spatial interview environment to mitigate interview anxiety, (2) extract candidate information from a PDF resume and inject it into conversational agents at runtime, (3) orchestrate three sequential 200 s interview stages (600 s total), (4) propagate earlier-stage transcripts and active follow-up context to subsequent interviewers, (5) measure head orientation, active gaze alignment within 35°, rotational posture stability and response latency at 1 Hz, (6) synthesise transcripts and telemetry into a ten-competency placement evaluation and vector PDF report, and (7) synchronize completed interview metrics to a centralized placement dashboard for institutional faculty monitoring. The questions the implementation answers are design questions:

- **DQ1.** Can resume-derived context be supplied to cloud conversational characters at runtime?
- **DQ2.** Can a three-persona panel be orchestrated sequentially with transcript forwarding?
- **DQ3.** Can gaze-alignment, posture and latency metrics be derived from HMD pose alone?
- **DQ4.** Can transcripts and telemetry be converted into a structured, exportable report automatically?

Four evaluation questions follow from the design but are not answered here: whether an immersive panel elicits stronger presence and realistic anxiety than a 2D conversational agent (EQ1); whether resume grounding improves perceived question relevance (EQ2); whether a specialised sequential panel gives more comprehensive preparation than a single general agent (EQ3); and how closely HMD-derived metrics agree with expert recruiter assessments (EQ4).

V. SYSTEM ARCHITECTURE

The system has four execution tiers (Fig. 1). The Unity client renders the environment, manages scenes and the interview state machine, captures 1 Hz telemetry, and runs the client feedback engine. A local Python subsystem, invoked as a headless child process, extracts resume text and renders the final vector PDF report. The Convai cloud performs speech recognition, language-model dialogue, speech synthesis and lip-sync generation; the Unity client communicates with it through the Convai Unity SDK. Finally, an institutional Placement Data System (Node.js/Express REST API on port 5000, PostgreSQL database via Prisma, and a Next.js 14 Placement Dashboard on port 3000) provides centralized cohort analytics, receiving interview payloads from Unity via PlacementBackendClient.

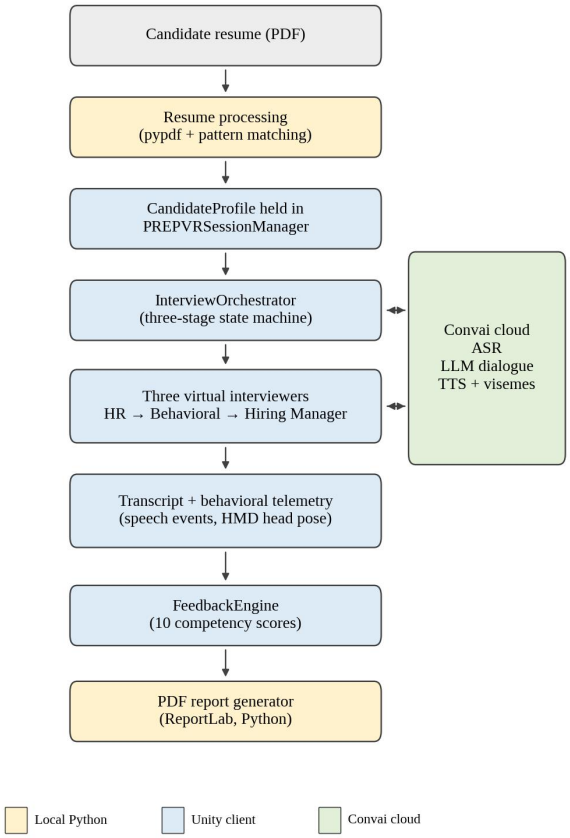


Fig. 1. Overall architecture. Shading indicates the execution tier: local Python, Unity client, Convai cloud.

TABLE II
TECHNOLOGY STACK (FROM PROJECT MANIFESTS)

Component	Technology (version)	Role
Engine	Unity 6 (6000.3.21f1); URP 17.3.0	Rendering, scenes, game logic
XR	XR Interaction Toolkit 3.6.1; XR Plugin Management 4.7.0; OpenXR	HMD tracking, controllers
Input	Input System 1.20.0	Keyboard, mouse, XR input
Conversation	Convai Unity SDK 4.6.0	Characters, audio streaming, dynamic context

Component	Technology (version)	Role
Language	C# (.NET Standard 2.1); Python 3.10+	Client logic; local scripts
Python libs	pypdf; reportlab	Resume text extraction; PDF rendering
UI	uGUI; TextMeshPro 3.2.0	Screens and runtime HUD
Placement System	Node.js 20, Express, Prisma, PostgreSQL 15; Next.js 14	Central database, REST API (port 5000), faculty dashboard (port 3000), institutional ranking
Placement Client	PlacementBackendClient.cs; FeedbackFormOpener.cs	Unity-to-backend REST synchronization, Google Forms URL integration

OpenXR is the Khronos open standard for cross-platform access to VR and AR devices [9].

The application is organised as seven scenes: Welcome, Home, Interview Type, Resume Upload, Resume Review, Interview Room and Interview Over. A persistent session manager (`PREPVRSessionManager`, a `DontDestroyOnLoad` singleton) carries the candidate profile, transcript, visual observations, feedback and an `IsVRMode` flag across scene transitions, and `SceneLoader` selects desktop or VR mode before entering the interview. The interview-type screen offers the HR and behavioural panel track; the technical coding track is displayed as "Coming Soon" and is not implemented.

VI. RESUME INTELLIGENCE AND PERSONALISATION

The resume pipeline (Fig. 2) begins when the candidate selects a PDF through a native Windows file dialog (the Unity Editor file panel is used in the editor). `ResumeUploadController` calls `PythonBackendBridge`, which launches `unity_bridge.py process_resume <file.pdf>` as a headless process and reads its standard output. The script extracts raw text with `pypdf` (falling back to binary stream scanning if needed) and applies deterministic pattern matching: regular expressions for e-mail addresses and telephone numbers, a lexicon of more than 45 technical keywords (for example Python, Java, C#, Machine Learning, Unity, React and Cloud) for skills, and keyword scans for education (University, College, Institute, B.Tech, M.S. and similar) and experience (Intern, Engineer, Developer, Lead and similar). The output is a structured JSON object containing `candidate_profile` (name, education, categorized technical/soft/tool skills, projects, experience, certifications, achievements) and `interview_context` (specialization, focus, starter question), which Unity deserialises into a `CandidateProfile`.

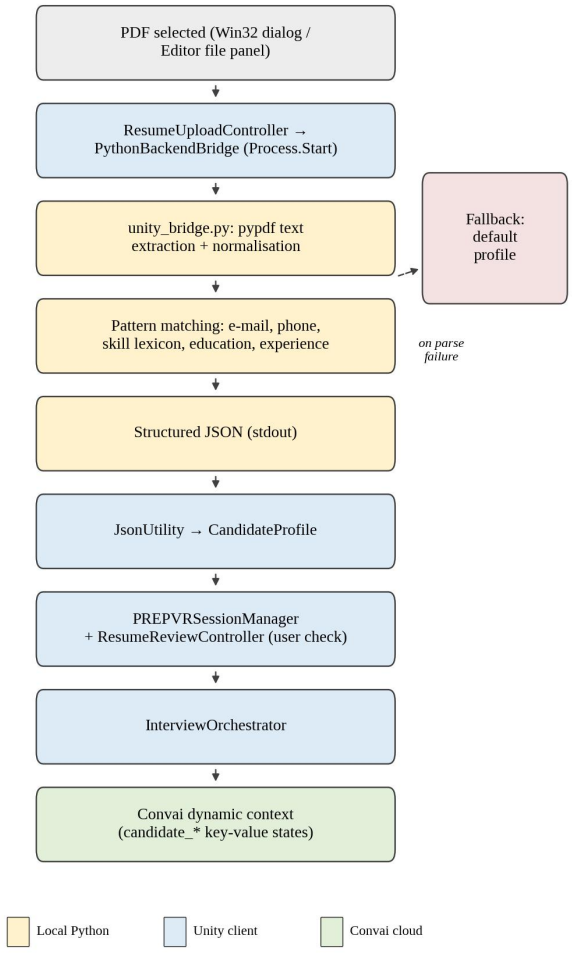


Fig. 2. Resume intelligence pipeline from PDF to Convai dynamic context.

The candidate reviews the parsed profile on the Resume Review screen before entering the room. If parsing fails because the PDF is corrupted or the Python environment is missing, the controller reports the failure and continues with a default profile so that the session does not fail.

At room initialisation the orchestrator supplies profile fields as key-value states to each character's dynamic context (`stage`, `stage_role`, `candidate_name`, `candidate_skills`, `candidate_education`, `candidate_projects`, `resume_context`, and opening `instructions`) and builds opening prompts that reference them, for example an instruction welcoming the candidate and asking about their background. Convai documents dynamic context as session-specific information pushed to a character at runtime that exists only for the current session and takes effect asynchronously, visible in the character's next reply [10]. The effect of grounding on question relevance has not been measured (EQ2).

The parser is heuristic. It uses keyword and pattern matching and not a trained named-entity-recognition or layout-aware model of the kind surveyed in [5]; unconventional or multi-column layouts may yield incomplete extraction.

VII. CONVERSATIONAL AI AND MULTI-INTERVIEWER ORCHESTRATION

A. Division of Responsibilities

Table III separates what runs locally from what runs in the cloud. Python does not call any external large-language-model service; resume parsing and the fallback evaluator in the Python script are deterministic. All generative dialogue is performed within Convai's hosted backend. The specific language models used by that backend are not exposed to the client and are not known to the authors.

TABLE III
EXECUTION RESPONSIBILITIES

Tier	Responsibilities
Unity (local)	Scenes, XR rig, orchestration, microphone capture and streaming via SDK, audio playback, lip-sync rendering, telemetry, feedback scoring
Python (local)	PDF text extraction and pattern-based parsing; PDF report rendering; a heuristic fallback evaluator
Convai (cloud)	Speech recognition, dialogue generation from persona prompt, narrative configuration and injected context, speech synthesis, viseme generation

B. Three-Stage Sequential Panel

The orchestrator, `InterviewOrchestrator`, is a deterministic finite-state machine over the stages Uninitialized, Ready, Stage1_HR, Stage2_Behavioral, Stage3_HiringManager and Complete (Fig. 3). Each stage targets 200 s (0:00–3:20 for HR, 3:20–6:40 for Behavioral, and 6:40–10:00 for Hiring Manager), totaling 600 s (10 minutes). A stage hands over when its timer exceeds 200 s or an exchange quota is met. A global 600 s interview timer runs in parallel, and either panel completion or timer expiry leads to the Interview Over scene. Table IV summarises the roles. The three interviewers are separate Convai characters; the HR interviewer uses a Convai sample avatar (Sofia) and the other two use a seated formal avatar.

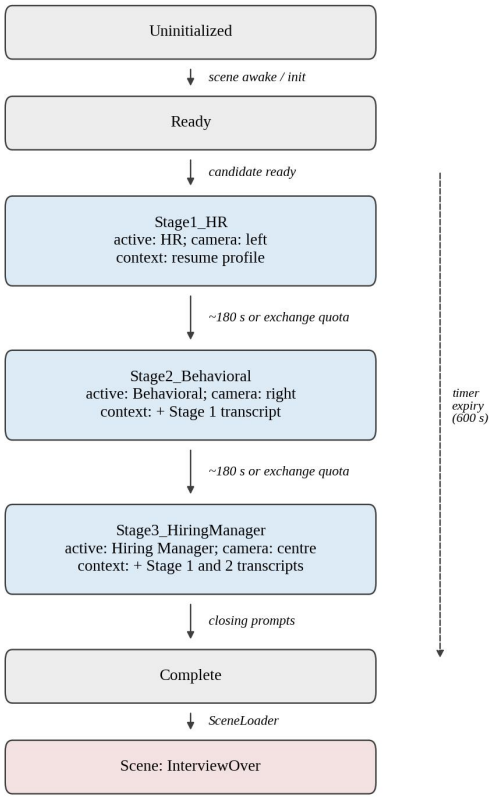


Fig. 3. Three-stage interview state machine with active target, camera focus and forwarded context per stage.

TABLE IV
INTERVIEW STAGES

Stage	Role (object name)	Focus	Context received
1 (200 s)	HR (HR Interviewer)	Background, resume, career trajectory, communication, motivation	Resume profile, skills, projects, opening instructions
2 (200 s)	Behavioural (Behaviour Analyst)	Teamwork, adaptability, problem solving; STAR-oriented questions	Profile + Stage 1 transcript + follow-up directives
3 (200 s)	Hiring Manager (Technical Interviewer)	Project execution, ownership, leadership, workplace readiness, candidate questions	Profile + Stages 1 & 2 transcripts + managerial instructions

Object names are those used in the Unity scene; the paper uses role names.

C. Conversational Interaction Loop

The verbal loop is shown in Fig. 4. The candidate's speech is captured by the microphone (HMD or desktop) and streamed by the SDK over a LiveKit-based channel to Convai, where it is transcribed and passed to the dialogue engine together with the character's persona and dynamic context. The response is returned as synthesised audio and viseme data, which the `ConvaiCharacter` component plays through a spatial `AudioSource` and maps to facial blend shapes. The orchestrator can also send text prompts

to the active character, which allows the interviewer to speak first at the start of each stage.

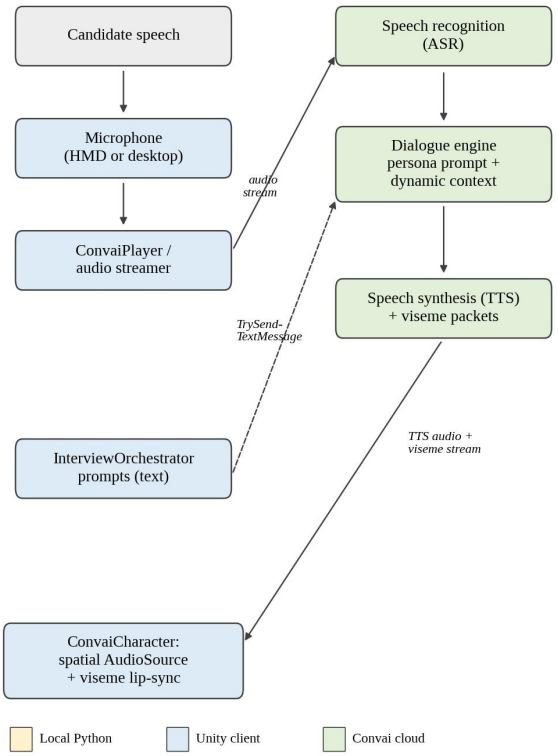


Fig. 4. Candidate conversational interaction loop. Dashed arrow: orchestrator prompts.

D. Active-Speaker Management and Context Propagation

In a room with three avatars, the system must determine which character receives the candidate's speech. The orchestrator implements the SDK's conversation-target interface, sets the player's targeting mode to manual, and updates the active character at each stage change; `ConversationTargetEnforcer` keeps the target on the active interviewer. At each handover the outgoing interviewer speaks a short hand-over line, `InterviewCameraController` turns the view toward the next interviewer, and the orchestrator injects the accumulated transcript lines into the incoming character's dynamic context (a previous-stage summary for Stage 2 and the concatenated transcripts of Stages 1 and 2 for Stage 3) before prompting it to begin. Furthermore, after each candidate response, the orchestrator updates character context dynamically via `SetStates(dynamicState, ConvaiRespondMode.Silent)` and flushes the state, allowing the AI to formulate grounded follow-up probes based on the candidate's actual verbal answers.

This is a *sequential* multi-agent architecture: one interviewer is active at a time, and transitions are coordinated by the orchestrator. Interviewers do not converse with one another, and natural interruptions or overlapping speech are not supported. The panel therefore approximates the handover structure of a multi-stakeholder interview but not its conversational dynamics.

VIII. VR/XR INTERVIEW ENVIRONMENT

The environment is a three-dimensional boardroom built in Unity 6 with the Universal Render Pipeline. VR support uses the XR Interaction Toolkit over OpenXR [9]. The rig consists of an XR Origin placed at (0, 0, −1.5), a camera offset, a main camera driven by an OpenXR TrackedPoseDriver that follows the HMD, and left and right controllers with ray interactors for interface interaction. The table is positioned at (0, 1.2, 0), and the three interviewers are seated at z = 2.0 with x = −1.5 (HR), 0 (Hiring Manager) and +1.5 (Behavioural), rotated 180° about the vertical axis to face the candidate (Fig. 7). Each interviewer is therefore a spatially distinct source of speech played through a 3D AudioSource. In Desktop mode, InterviewCameraController executes a smooth spherical linear interpolation (Quaternion.Slerp over 1.5 s) towards the active interviewer's face/chest level to simulate dynamic broadcast framing. In VR mode, the OpenXR TrackedPoseDriver continuously maps the user's physical HMD orientation directly to the camera, allowing natural visual orienting toward whichever avatar is speaking, while 3D spatial audio attenuation provides directional localisation.

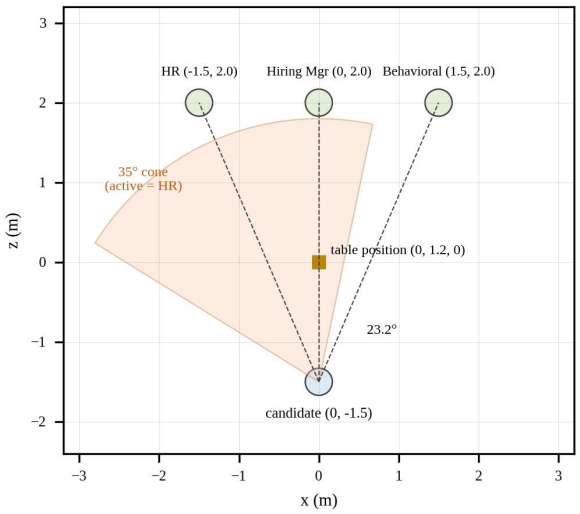


Fig. 7. Top-down layout from the scene coordinates, with the 35° gaze-alignment cone drawn about the direction to the HR interviewer. The marker is the table position; its extent is not recorded.

A runtime HUD shows the current stage, a countdown and the active speaker's name. In VR mode it is a world-space canvas at (0, 1.8, 1.8) in front of the candidate; in desktop mode it is a screen-space overlay. Desktop mode is selected from the home screen and provides a fallback when no headset is used.

The spatial design is motivated by the presence argument in [6]: an interview panel is a social situation in which place and plausibility may contribute to realistic responses. This is a design objective. Whether the environment produces stronger presence or more realistic interview anxiety than a conventional interface is untested (EQ1), and [7] concerns clinical exposure therapy and does not establish it.

IX. NON-VERBAL BEHAVIOURAL ANALYSIS

`InterviewVisualEvaluator` samples three non-verbal metrics at a discrete 1 Hz frequency (sampleTimer >= 1.0 s) rather than per frame. For gaze alignment, let P_c be the camera position, F its forward

unit vector and P_i the position of the active interviewer. The direction to the interviewer and the angular deviation are

$$D = (P_i - P_c) / \|P_i - P_c\| \quad (1)$$

$$\theta = \arccos(F \cdot D) \quad (2)$$

computed in Unity as `Vector3.Angle` between the camera's forward vector and the direction to the interviewer. A frame with $\theta \leq 35°$ is classified as eye contact, and a frame with $\theta > 35°$ as gaze aversion. The tracking-loss case is handled by clamping angular deltas so that no undefined values propagate.

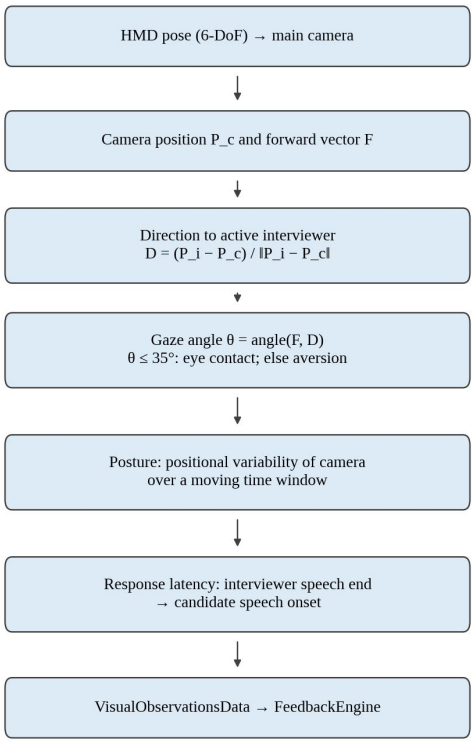


Fig. 5. Non-verbal telemetry pipeline from HMD pose to evaluation.

Posture stability is derived from the angular variability of the camera's forward orientation Vector3.Angle(lastHeadForward, currentForward) accumulated across 1-second sample intervals. The average angular velocity over the session is evaluated, with values below 5.0°/s classified as composed, stable seated posture. **Response latency** is the elapsed time between the completion of an interviewer's speech clip (RecordQuestionEnded) and the onset of candidate speech (RecordAnswerStarted). The evaluation model treats latency of 1.5 to 3.0 s as optimal thoughtful pacing, less than 0.5 s as premature interruption, and more than 6.0 s as excessive hesitation (Section X).

Head pose is not eye tracking. Eq. (2) uses the orientation of the headset, not the direction of the eyes. It estimates whether the candidate's head is turned toward the active interviewer, and a candidate could look elsewhere with their eyes while keeping the head still, or the reverse. Head orientation has been reported to carry much of the gaze signal in meetings [8], but that result concerned a different setting and does not validate Eq. (2) here. Hardware-based eye tracking was not used.

Consequence of the threshold in this room. With the candidate seated at (0, 0, −1.5) and the side interviewers at x = ±1.5, z = 2.0, the angular deviation to either side interviewer is arctan(1.5 / 3.5) ≈ 23.2°, while the two side interviewers subtend 46.4°. The 35° half-angle cone was chosen as an empirical design boundary: it encloses the active interviewer (23.2° < 35°) with adequate tolerance for natural head gesture inflection while discriminating against orienting towards the opposite side interviewer (46.4° > 35°). A candidate facing the central interviewer is within the tolerance cone of either side interviewer, but a candidate looking toward the left interviewer is classified as averted when the right interviewer speaks. Hardware-based pupil eye tracking was not used.

X. EVALUATION AND FEEDBACK SYSTEM

A. Candidate Placement Competency Model & Scoring Engine

After the interview, `FeedbackEngine` evaluates candidate performance across ten core placement competencies (Table V and Fig. 6), synthesizing verbal transcripts and non-verbal observations. Rather than arbitrary scoring, each competency receives an evidence-grounded qualitative rating tier ('Exemplary', 'Proficient', 'Developing', 'Needs Focus') with factual observations and actionable recommendations. For institutional reporting, these tiers map to numerical values (9.5, 8.5, 7.0, 5.5). In addition, Overall Readiness (0–100 scale) is computed as an explicit weighted composite: 40% overall competency average, 20% communication structure and listening, 20% behavioral ownership and uncertainty handling, and 20% hiring-manager technical and project depth.

TABLE V
PLACEMENT COMPETENCY MODEL

Competency	Source	Evaluation logic
Resume Alignment & Authenticity	Extracted profile + transcript	Keyword and technical tool overlap between stated resume skills/projects and spoken explanations.
Project Depth & Architecture	Stage 3 transcript lines	Comprehension of core system architecture, data flow, pipeline interfaces and database trade-offs.
Problem-Solving Methodology	Stage 2 transcript lines	Structured approach when discussing performance bottlenecks, debugging procedures and library choices.
Technical Fluency & Skills	Stage 2–3 transcripts	Fluent use of technical terminology relating to APIs, concurrency and system engineering.
Candidate Ownership	Stage 2–3 transcripts	Clear distinction of individual code contributions from team efforts using assertive 'I built' phrasing.
Handling Uncertainty	Stage 2–3 transcripts	Poise when challenged with edge cases; honest acknowledgement of limitations with reasoned mitigations.

Competency	Source	Evaluation logic
Continuous Learning	Stages 1–3 transcripts	Evidence of self-directed learning in emerging frameworks and code refactoring for maintainability.
Communication Clarity (STAR)	Verbal transcripts	Structure of scenario answers anchored to Situation, Task, Action, and quantifiable Result.
Professional Presence	Head pose & posture telemetry	Composed upright posture (angular drift < 5°/s) and visual focus (35° cone) during questioning.
Active Listening & Latency	Response onset latency	Attentive listening with measured response latency (1.5–3.0 s optimal; < 0.5 s interrupting; > 6.0 s hesitation).
Overall Placement Readiness	Weighted composite model	Weighted composite: 40% overall competency average + 20% communication + 20% behavioral + 20% hiring manager.

The engine also generates three actionable analytical lists: verified strengths, priority growth areas, and actionable recommendations. `InterviewOverController` presents the readiness score, qualitative tier, strengths and improvements on screen. On export, `FeedbackPdfGenerator` serialises the feedback data as JSON and invokes `generate_feedback_pdf.py`, which renders a multi-page vector PDF report with ReportLab, including score tables, colour-coded badges, action lists and 'Page X of Y' numbering.

B. Institutional Placement Department Web Portal (Placement Central)

To resolve the institutional scaling bottlenecks identified in the empathy study (Persona Sherin), the simulation synchronizes all completed session metrics with a dedicated institutional web portal (`placement-dashboard`) built with Next.js 14, React, TypeScript, and Tailwind CSS. The web portal completely decouples the placement department from the Unity VR client, ensuring that placement directors, faculty coordinators, and institutional leadership can review student performance without operating VR hardware. Key capabilities of the placement portal include:

• Secure Staff Authentication: Role-based faculty authentication (`admin@prepvr.edu`) restricts access to sensitive student performance records while offering demo autofill for rapid administrative inspection.
• Institutional KPI Overview Cards: An executive summary bar aggregates real-time cohort statistics: Total Enrolled candidates, Completed VR Tests (with institutional completion percentage), Active and Pending Headset Sessions, Cohort Average Readiness Percentage, and Top Candidate vs. Cohort Mean Scores.
• Multi-Department Directory and Ranking: The candidate directory supports multi-department filtering across Computer Science (CSE), Information Technology (IT), Artificial Intelligence & Data Science (AI&DS), Electronics (ECE), and Mechanical Engineering. Faculty can filter by interview status (`COMPLETED`, `PENDING`, `IN_PROGRESS`, `FAILED`), sort dynamically by institutional rank or readiness score, and view visual score progress bars.
• Candidate Deep-Dive Analytics Modal: Selecting an enrolled student opens an interactive analytics view displaying candidate metadata, an Institutional Readiness Tier Badge (e.g., *Placement Ready — Exemplary Tier 1*), five core placement competency gauges (Communication, Confidence, Professionalism, Behavioral, Technical), a 10-minute 3-stage panel breakdown (question counts, answer lengths, and durations across HR, Behavioral, and Hiring Manager stages), qualitative strengths and growth roadmaps, observable demeanor logs (visual gaze engagement % within the 35° cone, rotational posture stability velocity, and response latency), PDF report download/preview, and multi-round attempt history.
• Cohort Enrollment and External Survey: Placement coordinators can register student roll numbers directly into the PostgreSQL database, and `FeedbackFormOpener` provides a direct link to an external Google Forms feedback survey (`https://forms.google.com`) to capture candidate feedback directly after headset removal.

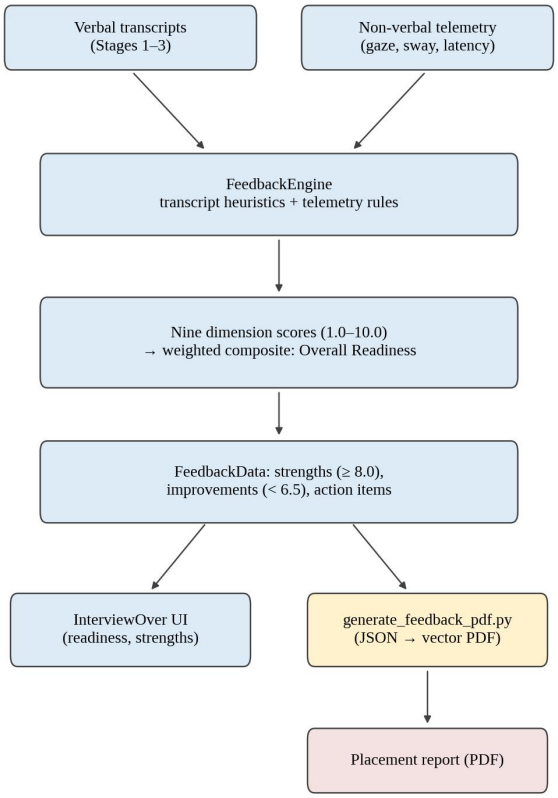


Fig. 6. Feedback generation architecture.

This scoring model is an engineering design and has not been validated against recruiter judgement or against hiring outcomes. Its thresholds, bands and composite weights are design parameters, and keyword-based indicators may reward vocabulary rather than answer quality. Fairness and interpretability concerns identified for automated interview scoring in general [5] therefore apply, and the output should be read as formative practice feedback and not as a selection decision.

XI. IMPLEMENTATION

The application is written in C# with two Python scripts. Only the classes that explain the architecture are described here.

Session and scene control. `PREPVRSessionManager` is the persistent singleton holding the profile, transcript, observations, feedback and mode flag, with convenience methods for recording answers and observations. `SceneLoader` wraps scene transitions and sets desktop or VR mode.

Resume path. `ResumeUploadController` opens the file dialog (Win32 `comdlg32.dll` in standalone builds) and coordinates parsing. `PythonBackendBridge` starts Python through `System.Diagnostics.Process`, passes arguments on the command line and reads JSON from standard output; `unity_bridge.py` performs extraction. `ResumeReviewController` presents the parsed profile before the room is entered. Temporary JSON files for the bridges are written to the application's persistent data path and overwritten per session.

Interview runtime. `InterviewOrchestrator` (about 1,500 lines of C#) holds the stage state machine, initialises the Convai player and the three characters, injects dynamic context, dispatches prompts and records transcript lines from speech-completed events. `ConversationTargetEnforcer` keeps the active target consistent, `InterviewCameraController` pans the view between interviewers, `InterviewTimerManager` runs the 600 s interview timer and 200 s stage timers, `InterviewRuntimeHUD` renders the dual-mode HUD, and `InterviewVisualEvaluator` produces the 1 Hz telemetry. A debug overlay, `InterviewTestMonitor`, shows transcripts and states at runtime.

Feedback and Placement Web Portal Architecture. `FeedbackEngine` evaluates the ten placement competencies into qualitative tiers, `InterviewOverController` presents the results on screen, and `FeedbackPdfGenerator` invokes `generate_feedback_pdf.py`. The placement web portal is implemented in the placement-dashboard/ codebase as a Next.js 14 App Router application. It interfaces with the Node.js/Express REST API (port 5000) via a typed API service (lib/api.ts), querying PostgreSQL 15 through Prisma schemas (Student, Interview, InterviewStage, FeedbackScore, ObservableDemeanor). The frontend consists of Navbar.tsx for staff authentication and navigation, SessionsView.tsx for real-time headset tracking, and StudentDetailsModal.tsx for deep-dive candidate analytics. In Unity, `PlacementBackendClient` formats and dispatches the completed interview payload via HTTP POST to /api/interviews upon session completion, updating institutional rankings and analytics without manual intervention. Finally, `FeedbackFormOpener` enables candidates to launch the post-interview survey in an external web browser.

Resilience. The Convai manager provides retry callbacks for lost connections, and the orchestrator logs a warning and keeps its state if a character does not respond within a timeout. Resume-parsing failure falls back to a default profile, and PDF-generation failure leaves the on-screen evaluation intact. Convai credentials are held in a Unity settings asset and are not reproduced here.

XII. TESTING AND SYSTEM VERIFICATION

The project includes an automated test harness, `InterviewAutomatedTester`, attached to the Interview Room scene. It listens to the microphone and cancels automatic injection if the human candidate speaks. If silence exceeds 4.5 s after an interviewer finishes, it injects a synthetic candidate response as a text message to the active character, drawn from a small pre-programmed corpus of three sample answers. This allows the full stage sequence to be exercised repeatedly without a human speaker in a headset. For the resume path, sample PDFs are kept in a test-resume folder and the parser can be run directly from the command line, which allows the extraction step to be checked outside Unity.

These mechanisms provide **system verification**: they exercise the pipeline and make runs repeatable. They are not **scientific validation**. Because answers are injected as text, the harness bypasses the microphone and speech-recognition path; its three fixed answers do not represent candidate diversity; and no pass or fail criteria, accuracy measures or run statistics are reported. Nothing in this section supports a claim about parsing accuracy, conversational quality, scoring validity or user benefit.

XIII. IMPLEMENTATION OBSERVATIONS

The project inspection confirms that the end-to-end simulation pipeline, resume intelligence parser, multi-agent orchestrator, non-verbal telemetry, and centralized placement data system are fully implemented and functionally verifiable. Institutional cohort performance is demonstrated through the seed database (7 student profiles across 5 engineering departments) and exported CSV directories. However, no clinical randomized human-subject trial comparing VR training with human recruiters has been conducted. This section records the verified implementation status found in the repository (Table VI).

TABLE VI
IMPLEMENTATION VERIFICATION MATRIX

Capability	Evidence (source)	Status
3D boardroom, three seated interviewers	Interview Room scene	Implemented
OpenXR rig; desktop fallback	Package manifest, scene, SceneLoader, HUD	Implemented
PDF upload and local parsing	ResumeUploadController, unity_bridge.py	Implemented
Parsed-profile review screen	ResumeReviewController	Implemented
Sequential three-stage orchestration with manual targeting	InterviewOrchestrator, ConversationTargetEnforcer	Implemented
Cross-stage transcript forwarding	InterviewOrchestrator	Implemented
Convai integration	Convai settings asset, ConvaiCharacter	Configured and verified
Gaze, sway, latency telemetry	InterviewVisualEvaluator	Implemented
Ten-competency model	FeedbackEngine	Implemented

Capability	Evidence (source)	Status
Vector PDF report	generate_feedback_pdf.py	Implemented
Automated test harness	InterviewAutomatedTester	Implemented
Technical coding track	InterviewTypeController	Proposed
Interviewer-to-interviewer dialogue	InterviewOrchestrator	Proposed
Centralized Placement Data System & Dashboard	Backend (Express/Prisma/PostgreSQL), placement-dashboard (Next.js 14)	Implemented
Unity-to-Backend REST Client & Feedback URL	PlacementBackendClient.cs, FeedbackFormOpener.cs	Implemented
Direct Python LLM SDK Calls (OpenAI/Gemini)	Backend/resume_intelligence/unity_bridge.py	Not implemented (Handled via Convai cloud)

The inspection shows that an end-to-end path exists from resume upload through a three-stage voice interview to an exported report and centralized database synchronization. It also shows that session state persists across seven scenes through a single manager, and that evaluation operates on recorded transcripts and observations independently of real-time audio playback. The institutional placement portal (Next.js on port 3000) visualizes department performance, student rankings, and competency breakdowns. These are observations about the code, database schema, and system configuration; they confirm architectural completeness while distinguishing functional implementation from clinical human-subject efficacy.

XIV. LIMITATIONS

A. Technical

- **Cloud dependence.** Recognition, dialogue and synthesis occur in Convai's cloud, so turn-around latency depends on network conditions and remote queueing. It has not been measured.
- **Heuristic resume parsing.** Extraction relies on patterns and keyword lexicons and may be incomplete for unconventional or multi-column layouts.
- **Sequential dialogue.** One interviewer is active at a time; interruptions, overlapping speech and interviewer-to-interviewer exchange are not supported.
- **Platform dependence.** The standalone file dialog uses the Windows Win32 API, and parsing and report generation require a local Python environment with the needed libraries. Other platforms are unverified.
- **Head-pose gaze estimate.** Eye contact is estimated from headset orientation, not from eye tracking, using an undocumented 35° threshold.
- **Undisclosed models.** The language models behind Convai and the configuration of its cloud-side dialogue graphs are not visible from the repository.

B. Validation and Threats to Validity

- Absence of raw survey response sheets and clinical trials. While the human-centred empathy study, personas (Monisha, Sherin), and user pain points are documented in project design materials, individual Google Forms survey response spreadsheets and raw interview audio recordings are unavailable in the project archive. Furthermore, no large-scale randomized controlled trial comparing VR with human mock interviewers has yet been completed; claims remain strictly anchored to implementation verification and functional testing.
- **Unvalidated scoring.** Thresholds, bands and weights are design parameters, and the scoring has not been checked for bias or reliability.
- **Hardware and session records.** The project configures OpenXR but the material used for this paper does not record the headset model or the duration of VR sessions, so no hardware-specific claim is made.
- **Telemetry interpretation.** The automatic camera panning toward the active interviewer may interact with HMD-driven head pose, and the interpretation of the metrics in desktop mode is not characterised. The speech-onset detection and the sway window are implementation details that should be documented for reproducibility.
- **Limited test scope.** The harness uses three fixed text responses and bypasses speech recognition (Section XII). The system has not been deployed at scale.

C. Privacy and Data Handling

The application keeps profile, transcript and observation data in memory and does not upload them to its own database. However, the architecture sends microphone audio to the Convai cloud and injects resume-derived fields and earlier-stage transcript lines into character context, so personal information does reach a third-party service. Use with real candidate data would require informed consent and review of the provider's data-handling terms, and the fairness and privacy concerns noted for AI-based assessment [5] apply.

XV. FUTURE WORK

Planned in the project. A technical coding-interview track (currently displayed as "Coming Soon") and more natural multi-interviewer interaction in place of sequential hand-over.

Proposed by the authors. (i) User studies addressing EQ1 to EQ4 with validated instruments and appropriate controls; (ii) recruiter validation of the competency model and calibration of the thresholds and weights; (iii) richer adaptive questioning based on the candidate's answers; (iv) transformer- or LLM-based resume extraction in place of the heuristic parser, with extraction accuracy evaluated on annotated resumes [5]; (v) hardware eye tracking to replace or validate the head-pose estimate; (vi) characterisation of camera panning against HMD pose; (vii) cloud persistence and multi-user support; (viii) cross-platform file handling; and (ix) privacy-preserving options such as redacting identifying fields before they are sent as context.

XVI. CONCLUSION

This paper described VR Interview Simulation, an immersive mock-interview platform grounded in a human-centred design thinking empathy study that addresses student interview anxiety and placement cell scalability constraints. The system integrates a Unity and OpenXR environment, resume-grounded cloud conversational AI, a sequential three-interviewer panel with transcript forwarding across three 200 s stages, 1 Hz head-pose behavioural telemetry, an automated ten-competency feedback pipeline producing vector PDF reports, and an institutional placement web dashboard for cohort tracking. Implementation status was verified through inspection of the project. The principal open questions are empirical: whether the immersive, resume-grounded panel improves presence and perceived relevance, and whether the heuristic, head-pose-based scores agree with expert recruiter judgement. Answering them requires the user studies described in Section XV.

ACKNOWLEDGMENT

The authors thank Dr. P. Revathy for her guidance and support throughout this project. The authors also acknowledge the Department of Computer Science and Design, Rajalakshmi Engineering College, Chennai, Tamil Nadu, India, for providing the necessary academic support and resources.

REFERENCES

[1] J. Levashina, C. J. Hartwell, F. P. Morgeson, and M. A. Campion, "The structured employment interview: Narrative and quantitative review of the research literature," *Personnel Psychol.*, vol. 67, no. 1, pp. 241–293, 2014.

[2] M. J. Smith *et al.*, "Job offers to individuals with severe mental illness after participation in virtual reality job interview training," *Psychiatr. Serv.*, vol. 66, no. 11, pp. 1173–1179, 2015, doi: 10.1176/appi.ps.201400504.

[3] M. E. Hoque, M. Courgeon, J.-C. Martin, B. Mutlu, and R. W. Picard, "MACH: My automated conversation coach," in *Proc. ACM Int. Joint Conf. Pervasive Ubiquitous Comput. (UbiComp)*, Zurich, Switzerland, Sep. 2013.

[4] L. Hemamou, G. Felhi, V. Vandenbussche, J.-C. Martin, and C. Clavel, "HireNet: A hierarchical attention model for the automatic analysis of asynchronous video job interviews," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, 2019, pp. 573–581, doi: 10.1609/aaai.v33i01.3301573.

[5] C. Qin *et al.*, "A comprehensive survey of artificial intelligence techniques for talent analytics," arXiv:2307.03195 [cs.CY], v3, May 2025. [Online]. Available: https://arxiv.org/abs/2307.03195

[6] M. Slater, "Place illusion and plausibility can lead to realistic behaviour in immersive virtual environments," *Phil. Trans. R. Soc. B*, vol. 364, no. 1535, pp. 3549–3557, 2009, doi: 10.1098/rstb.2009.0138.

[7] M. B. Powers and P. M. G. Emmelkamp, "Virtual reality exposure therapy for anxiety disorders: A meta-analysis," *J. Anxiety Disord.*, vol. 22, no. 3, pp. 561–569, 2008, doi: 10.1016/j.janxdis.2007.04.006.

[8] R. Stiefelhagen and J. Zhu, "Head orientation and gaze direction in meetings," in *CHI '02 Extended Abstracts on Human Factors in Computing Systems*, Minneapolis, MN, USA, Apr. 2002, pp. 858–859.

[9] Khronos Group, "Khronos releases OpenXR 1.0 specification establishing a foundation for the AR and VR ecosystem," press release, Jul. 29, 2019. [Online]. Available: https://www.khronos.org/news/press/khronos-releases-openxr-1.0-specification-establishing-a-foundation-for-the-ar-and-vr-ecosystem

[10] Convai, "Dynamic context," Convai Unity SDK documentation. [Online]. Available: https://docs.convai.com/api-docs/plugins-and-integrations/convai-unity-sdk/features/dynamic-context (accessed Oct. 4, 2026).

Preparation notes for the authors

This page is not part of the paper. Delete it before submission.

1. Reference provenance

None of the ten references came from the original project materials. All were researched during preparation and checked against publisher, repository or institutional records. Please re-check the following before submission: the UbiComp 2013 paper [3] is cited without page numbers because they were not confirmed; [9] and [10] are web resources and [10] carries an access date of 4 Oct 2026; the sources for [2] and [4] were confirmed through abstracts and indexing records, so statements about those systems are limited to what the abstracts say (Table I marks everything else "NR").

2. Summary of Project Dossier Points and Codebase Cross-Referencing

- • Stage 1 Context & Command Line: Verified in PythonBackendBridge.cs and unity_bridge.py. The command passed is 'process_resume <file.pdf>' (not 'parse'). The dictionary injected into Stage 1 Convai dynamic context contains: stage ('1'), stage_role ('HR Interviewer'), candidate_name, candidate_education, candidate_skills, candidate_projects, resume_context, and opening instructions.
- • Posture Stability & Telemetry Sampling: Verified in InterviewVisualEvaluator.cs. Metrics are sampled at 1.0 s intervals (1 Hz), not every frame. Posture stability tracks rotational angular delta Vector3.Angle(lastHeadForward, currentForward) between 1 s samples; average angular velocity < 5.0°/s is evaluated as stable/composed posture.
- • Stage Duration & Response Latency: Verified in InterviewOrchestrator.cs and InterviewTimerManager.cs. Each stage targets 200 s (0:00–3:20 for HR, 3:20–6:40 for Behavioral, 6:40–10:00 for Hiring Manager), totaling 600 s (10 minutes). Response latency is timed between RecordQuestionEnded and RecordAnswerStarted, with 1.5–3.0 s optimal, < 0.5 s interrupting, and > 6.0 s hesitation.
- • Overall Readiness Weights: Verified in PlacementBackendClient.cs (lines 205–211). Readiness score (0–100 scale) is computed as: 40% Overall Competency Score + 20% Communication Score (Structure + Active Listening) + 20% Behavioral Score (Ownership + Uncertainty + Continuous Learning) + 20% Hiring Manager Score (Resume Alignment + Project Depth + Problem Solving + Technical Fluency).
- • Camera Panning vs HMD Pose: Verified in InterviewCameraController.cs. In Desktop mode, SmoothRotateCameraCoroutine executes a smooth Slerp over 1.5 s to focus the monitor view. In VR mode, the OpenXR TrackedPoseDriver maps physical HMD orientation directly, allowing natural visual orienting while 3D audio attenuation provides directional cues.
- • Centralized Placement System & Dashboard: Fully verified in Backend/ (Express, Prisma, PostgreSQL on port 5000), placement-dashboard/ (Next.js 14 on port 3000), and PlacementBackendClient.cs (POST /api/interviews). Institutional cohort data includes 7 realistic seed students across CSE, IT, Mechanical, AI&DS, and ECE, with exported CSV directories.
- • Empathy Study Evidence: Grounded in VR prep.pptx ('MS INNOVATE - DESIGN THINKING & INNOVATION'). Slides 6–10 contain detailed Empathy Maps for Monisha (Student) and Sherin (Placement Faculty), Pain & Gain analysis, and the 'How Might We' problem definition that drove the dual student-faculty architecture.
- • Missing Evidence / Gaps Identified: Raw Google Forms survey response spreadsheets (.csv/.xlsx) and verbatim interview audio recordings were not stored in the repository. FeedbackFormOpener.cs contains the configurable Google Forms link (https://forms.google.com).
- • Contact Provenance: Author e-mails confirmed as nslswetha2007@gmail.com and mukeshva95@gmail.com. Academic advisor: Dr. P. Revathy, Department of Computer Science and Design, Rajalakshmi Engineering College.

3. Material that would strengthen the paper

- Screenshots of the six views recommended in the dossier (resume upload, profile review, first-person room view, active interviewer, evaluation dashboard, exported report page).
- A sample exported PDF report and any run logs from the automated harness, which would allow Section XIII to describe observed runs.
- Any pilot feedback from users, even informal, reported strictly as informal feedback and not as results.